pfi

# Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

# Memory and State

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Today: memory and state

## Two kinds of memory, kept apart

| | |
|---|---|
| `Session` | a conversation: every message, sent again with the next question |
| `Desk memory` | values: each with its unit, date, source, and run |

## Today

1. What state one agent run has
2. The framework's session and a follow-up
3. desk_memory, remember, recall in SQLite
4. The stale number, and which check sees it
5. What to keep, and what not to
6. Memory and replay: history in the key
7. Context, two kinds, settings; Excel, Git, AI

**Every model response in the notebook is a saved response, replayed. A live run can differ.**

# The rules for agents

## On the curve: a level and a change in the file, described

### Allowed names

the 10 Yr par yield in the file

the change in bp between two dates

the slope in bp

### Never called

a forecast or an outlook

a signal

a view on rates

## On every agent's output: text, a saved run

### Allowed names

the agent returned

the saved run

the brief states

### Never called

the agent's opinion or view

what the agent believes

the agent's advice

# How the state of a run works

| | |
|---|---|
| **The problem** | A model keeps nothing between requests |
| **The list** | The system message, question, tool requests, results, and answer so far |
| **Every request** | Sends the whole list; a new run starts empty unless your code supplies the old one |
| **You control** | What goes into the list |
| **What comes out** | A run whose whole state is its messages |

# What state an agent has: the messages of one run

```
messages of the run: 5: ['system', 'user', 'assistant', 'tool', 'assistant']
to_input_list: 4 items: ['user', 'function_call', 'function_call_output', 'message']
messages in each request: [2, 4]
```

### Every request sends the whole list so far

system, the question, each tool request, each tool result, the answer: nothing else carries over

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 2,232 prompt and 128 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

**A new Runner.run starts from two messages again, unless your code sends the list.**

# How a session works

| | |
|---|---|
| **The problem** | A follow-up that says the same yield needs the first question |
| **The method** | The framework reads the session's items, sends them ahead of the new question, adds the new items |
| **The model sees** | One longer list, as if asked in a single conversation |
| **You control** | The session id, the file, and an optional limit on items read back |
| **What comes out** | A follow-up that works, at the price of a longer request every time |

# The framework's session: a follow-up that refers back

```
session = agents.SQLiteSession("desk", work / "session.db")
outcome = await agents.Runner.run(market_agent, FOLLOWUP_QUESTION, max_turns=8, session=session)
```

```
the same requests as the first run, key for key: yes
```

### The follow-up, "the same yield", answered

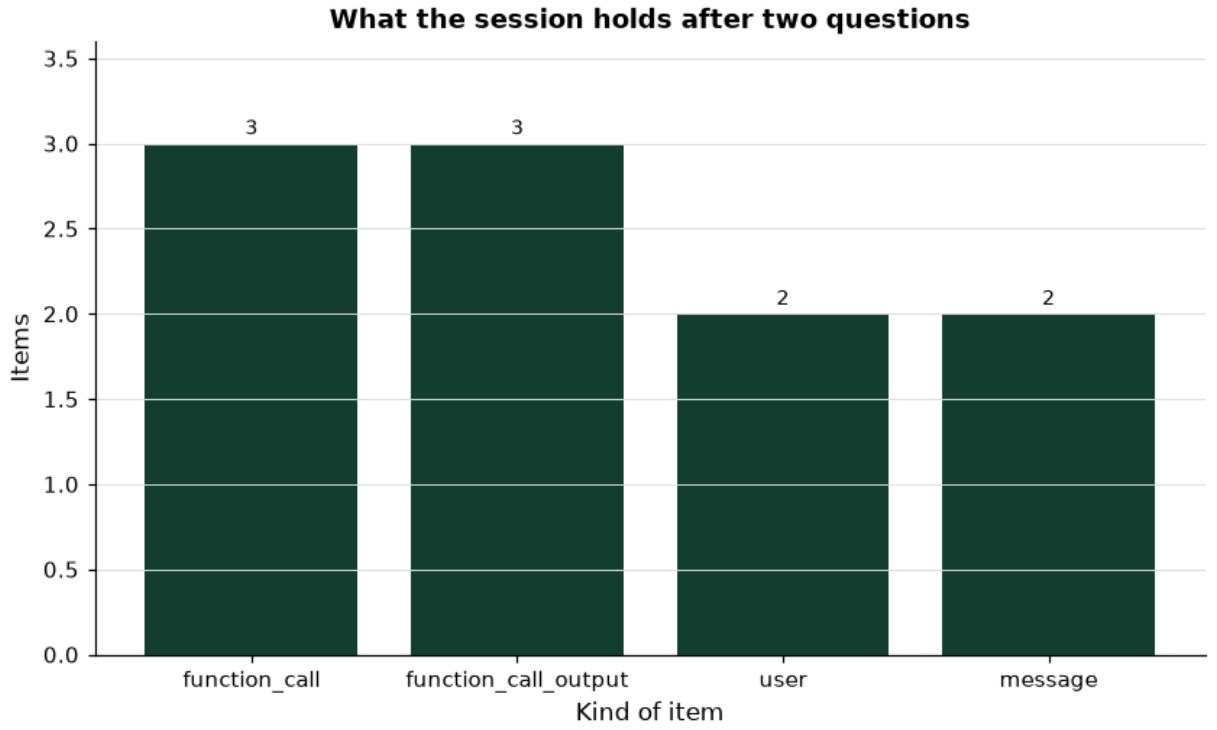
The 10 Yr par yield on 2026-10-01 was 5.24%. It increased by 4 basis points from 2026-10-01 to 2026-10-02.

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 4,284 prompt and 241 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

```
tables: ['agent_messages', 'agent_sessions', 'sqlite_sequence']
agent_messages columns: ['id', 'session_id', 'message_data', 'created_at']; rows for session 'desk': 10
```

**10 items in the session: both runs. created_at, the database's clock, is never printed.**

# What the session holds




**10 items after two questions; the next question sends every one of them again, ahead of the question.**

# How a desk memory works

| | |
|---|---|
| **The problem** | A desk keeps values, each true on a date, each from a source |
| **The row** | A number a tool returned, with unit, date, source, and run |
| **The access** | remember writes a row; recall matches the key and the exact date |
| **You control** | The key, the date, and the source on every write |
| **What comes out** | A traceable value, or the error result when the date is not there |

# A desk memory in SQLite

```
db = agentkit.desk_memory(work / "desk_memory.db")
# table memory: ['key', 'value', 'unit', 'as_of', 'source', 'run']
agentkit.remember(db, key, value, unit, as_of, source, run)
agentkit.recall(db, key, as_of)    # that date only, or the error result
```

```
key  value unit      as_of                          source          run
10 Yr par yield   5.24  pct 2026-10-01 treasury_yield(10 Yr, 2026-10-01) 22edaf3c04be
```

Course 3's habits: ? parameters, commit after each write, pd.read_sql_query to read back.

**Primary key (key, as_of): one value per key and date. 57 passed with the day's three tests.**

# How a stale number happens

| | |
|---|---|
| **The problem** | A value true once, offered for a date it was not true on |
| **The cause** | A memory that returns the latest value by default |
| **The stop** | The date in the key: recall returns a row only for the date asked |
| **The limits** | check_figures rejects a figure that cites no tool; numbers_untraced skips the question's own |
| **What comes out** | A refusal, or a number to check against its source |

# The stale number: which check sees it

```
recall 10 Yr par yield on 2026-10-02: {'error': 'no value for that date', 'key': '10 Yr par yield', 'as_of': ...
recall 10 Yr par yield on 2026-10-01: {'key': '10 Yr par yield', 'value': 5.24, 'unit': 'pct', 'as_of': '2026...
```

Desk memory: 10 Yr par yield 5.24 pct. What was the 10 Yr par yield on 2026-10-02 in the Treasury par curve file? A...

### Market agent: requested the tool

DESK_PROMPT's rule held: 5.28, the file's

check_figures: passes

### Memory desk, no tool: answered from it

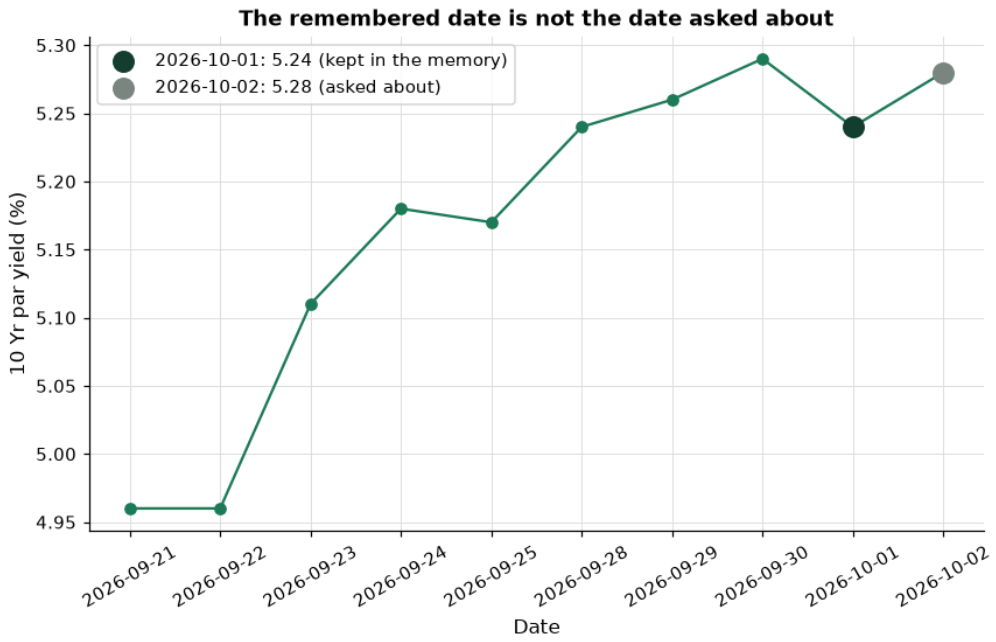
5.24, the 2026-10-01 value, for 2026-10-02

check_figures: rejected, cites no tool request

```
name  value unit  tool_value  cited_exists  value_matches  unit_matches  passed
10 Yr par yield on 2026-10-02   5.24  pct         NaN         False          False         False   False
```

**numbers_untraced found nothing in either: the number came in the question.**

# The date asked about




| Run | Stated | Traced |
|---|---|---|
| Market agent | 5.28 | True |
| Memory desk | 5.24 | False |
| Curve file | 5.28 | the file |

**The remembered date is not the date asked about. The memory desk's figure has no tool request behind it.**

# How keep works

| | |
|---|---|
| **The problem** | A memory is only as good as what goes into it |
| **The method** | Read a run's own tool requests and results; write a row for each value a tool returned |
| **Never kept** | A sentence the model wrote: it can round, mix dates, or restate a number no tool returned |
| **You control** | The keys, and which tool results are kept |
| **What comes out** | A table whose every row can be checked again against its file |

# What to keep: values with their date and source

```
values kept: {'first': 1, 'follow-up': 2}
                            key  value unit      as_of                                          source       ...
10 Yr change bp since 2026-10-01   4.00   bp 2026-10-02 curve_change_bp(10 Yr, 2026-10-01, 2026-10-02) 22edaf...
                10 Yr par yield   5.24  pct 2026-10-01              treasury_yield(10 Yr, 2026-10-01) 22edaf...
                10 Yr par yield   5.28  pct 2026-10-02              treasury_yield(10 Yr, 2026-10-02) df778b...
```

```
refused: ValueError, the value is not a number
rows before 3, after 3
```

Keep the tool's value, its unit, its date, its source, its run. Never the model's sentence.

**A change is true of two dates: kept on its end date, its start date in the key.**

# How memory and replay work

| | |
|---|---|
| **The key** | A saved response is stored under the hash of the whole request body |
| **The history** | A session puts its items into the body, so they are part of the key |
| **By hand** | Carrying the items yourself gives the same body, so the same key |
| **You control** | The history that goes in |
| **What comes out** | A replay that works only from the history it was recorded with |

# Memory and replay: the history is in the key

```
request 1: 6 messages, 1,324 prompt tokens, key 22edaf3c04be
request 2: 8 messages, 1,415 prompt tokens, key 5c36b2a9798d
request 3: 10 messages, 1,545 prompt tokens, key 1d6a2a42ad90
```

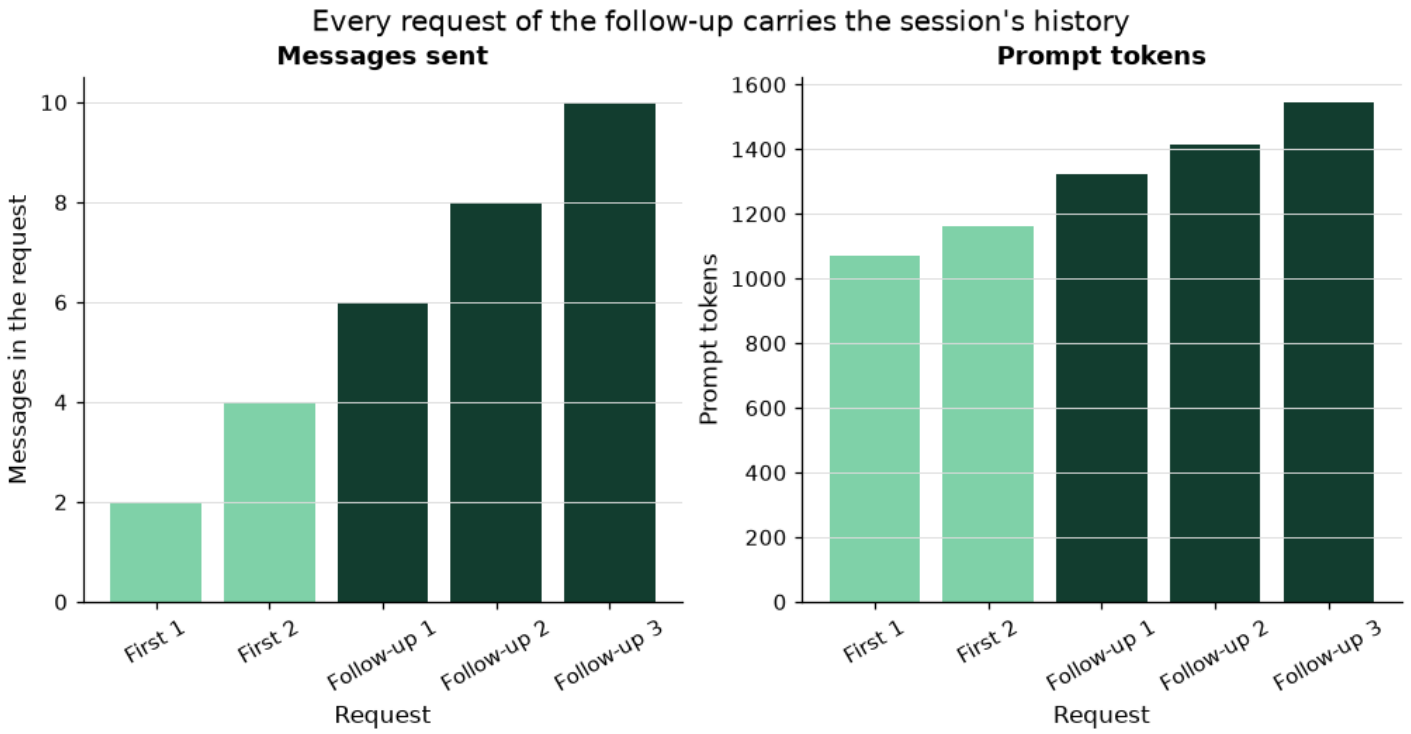
the same requests as the session's follow-up, key for key: yes

```
the follow-up alone: 2 messages, key 9c7898840645
the same key as the session's first follow-up request: no
```

**A session is the list, kept in a file. A session that grows between runs changes every later key.**

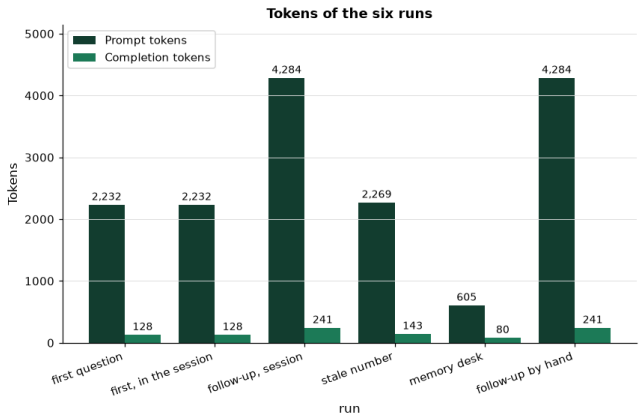
**The notebook's session starts empty in a fresh work folder, so the saved runs replay.**

# Every request carries the history




**Light bars: the first question. Dark bars: the follow-up. The session sends its history ahead of the question.**

# The runs of the day





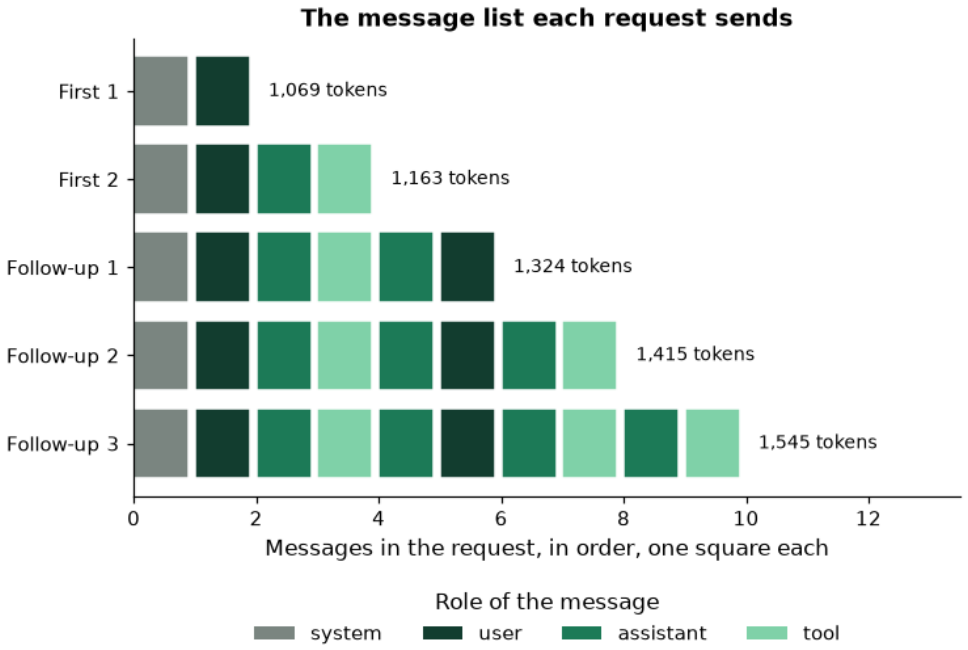


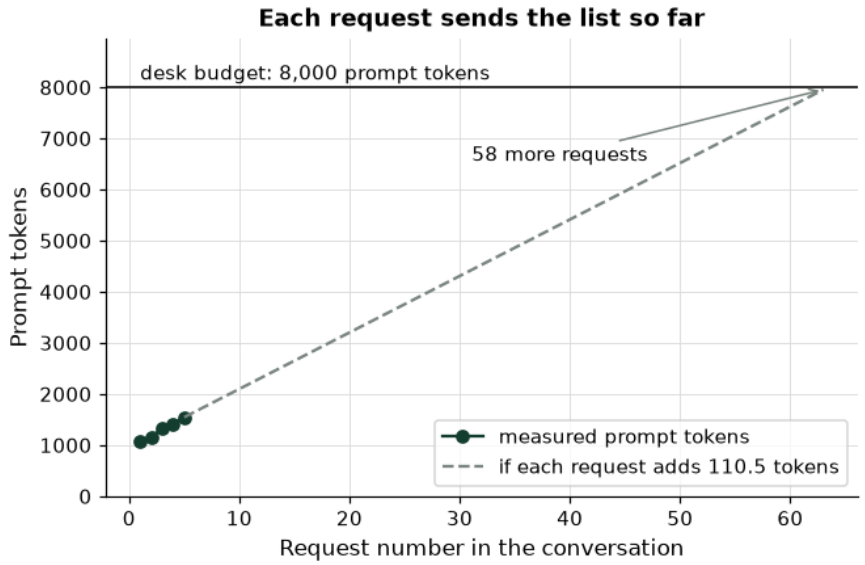
| Run | Requests | Prompt | Completion | Cost (US$) |
|---|---|---|---|---|
| first question | 2 | 2,232 | 128 | 0.001098 |
| first, in the session | 2 | 2,232 | 128 | 0.001098 |
| follow-up, session | 3 | 4,284 | 241 | 0.002099 |
| stale number | 2 | 2,269 | 143 | 0.001136 |
| memory desk | 1 | 605 | 80 | 0.000370 |
| follow-up by hand | 3 | 4,284 | 241 | 0.002099 |

# How the context window works

| | |
|---|---|
| **The problem** | A model reads a limited list in one request, and the messages fill it |
| **The growth** | Each turn adds the reply, the tool results, and the next question; all of it is sent again |
| **The effect** | Prompt tokens climb with every request |
| **You control** | Items a session reads back, what a tool returns, when a conversation ends |
| **What comes out** | A line of prompt tokens that will cross a budget you set |

# The message list grows



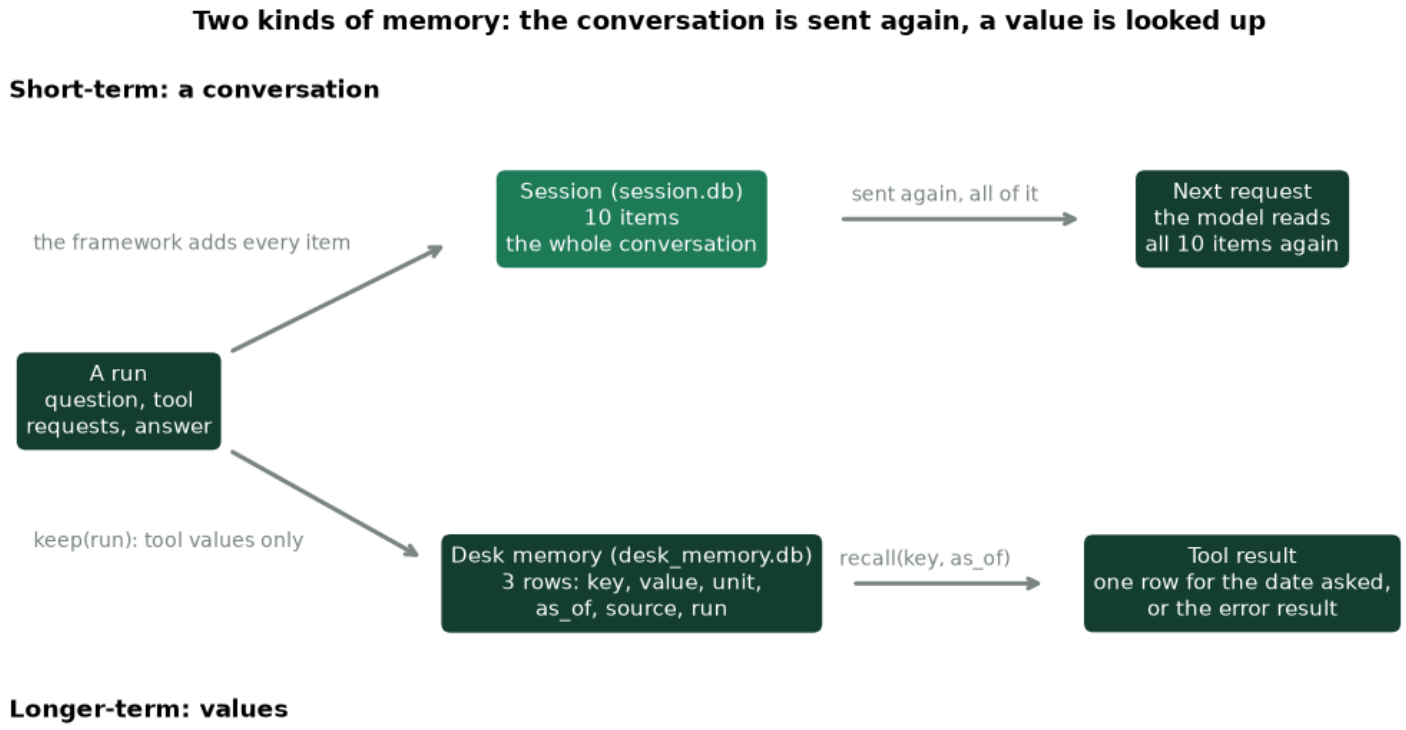




**Growth 110.5 prompt tokens a request: 58 more requests under the desk's own budget of 8,000. An estimate.**

# How the two kinds of memory differ

| | |
|---|---|
| **Short-term** | The conversation: a list the model reads in full with every request |
| **Longer-term** | A table your code keeps; the model reads only the rows a tool looks up |
| **The cost** | The first costs tokens on every request; the second costs a lookup and a rule |
| **You control** | Which facts go where |
| **What comes out** | The conversation holds what was said; the table holds what was true, on a date |

# Two kinds of memory




**The model reads every item of the session, and no row of the desk memory directly.**

# How the settings of a memory work

| | |
|---|---|
| **The session** | Its id, its file, and a limit on the items read back |
| **The desk memory** | The arguments of remember and recall: key, date, unit, source |
| **Where read from** | Each row is read from the installed code |
| **You control** | The ones marked tune first |
| **What comes out** | What can be set, what the course sets, and why |

# Every setting of the session

| Object | Setting | Default | The course sets | Tune first |
|---|---|---|---|---|
| `SQLiteSession` | `session_id` | `required` | desk | yes |
| `SQLiteSession` | `db_path` | `':memory:'` | session.db in the work folder | - |
| `SQLiteSession` | `sessions_table` | `'agent_sessions'` | - | - |
| `SQLiteSession` | `messages_table` | `'agent_messages'` | - | - |
| `SQLiteSession` | `session_settings` | None | - | - |
| `SessionSettings` | `limit` | None | - | yes |
| `RunConfig` | `session_input_callback` | None | - | - |
| `RunConfig` | `session_settings` | None | - | - |
| `get_items` | `limit` | None | - | - |
| `Runner.run` | `session` | None | the session, on the follow-up | - |

**Few settings. The course sets the id and the file. Tune first: the limit, when a conversation outgrows its budget.**

# Every setting of the desk memory

| Function | Argument | Default | The course sets | Tune first |
|---|---|---|---|---|
| `desk_memory` | `path` | `required` | desk_memory.db in the work folder | - |
| `remember` | `db` | `required` | yes | - |
| `remember` | `key` | `required` | yes | - |
| `remember` | `value` | `required` | yes | - |
| `remember` | `unit` | `required` | yes | - |
| `remember` | `as_of` | `required` | yes | yes |
| `remember` | `source` | `required` | yes | - |
| `remember` | `run` | `required` | yes | - |
| `recall` | `db` | `required` | yes | - |
| `recall` | `key` | `required` | yes | - |
| `recall` | `as_of` | `required` | yes | yes |

**Tune first: the date, which remember and recall both take. Without it a memory returns the stale number.**

# Excel: a lookup by key and date with XLOOKUP

```
=XLOOKUP(1,(A2:A4="10 Yr par yield")*(D2:D4="2026-10-02"),B2:B4,"no value for that date")
```

| | |
|---|---|
| key and date 2026-10-02 | 5.28 |
| key and date 2026-10-01 | 5.24 |
| key and date 2026-09-30 | no value for that date |
| key alone, first to last | 5.24 |
| key alone, last to first | 5.28 |

ISTEXT(D2) TRUE; a date typed into H1: ISTEXT FALSE, lookup returns "no value for that date"

**Excel 16.0 build 20430, typed by script. The two-column lookup is recall; the key alone is the AI check's bug.**

# Git: read the change back, ignore the side files

```
diff --git a/agentkit.py b/agentkit.py
@@ -1475,3 +1475,50 @@ def route_table(result) -> pd.DataFrame:
+import sqlite3  # the desk's memory, from day 12 (Course 3's habits)
+def desk_memory(path) -> sqlite3.Connection:
+def remember(db: sqlite3.Connection, key: str, value: float, unit: str, as_of: str, source: str, run: str)...
+def recall(db: sqlite3.Connection, key: str, as_of: str) -> dict:
```

```
?? session.db-shm
?? session.db-wal
```

```
%%writefile -a .gitignore      *.db-wal  *.db-shm      then: !! session.db-wal, !! session.db-shm
```

**The session opens SQLite in write-ahead-log mode; *.db does not match its two side files.**

# Exercises

1. Clear the session and ask the follow-up again: what does it send, and what comes back?
2. Keep the synthetic book's DTS total and recall it by date.
3. Write memory_dates(db, key) with a test.
4. Write requests_left(prompt_tokens, budget) with a test.
5. AI check: a recall that returns the most recent value.

**The solutions notebook has a new work folder: its session and its memory start empty.**

# The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

## Docstring

That date only, or the error result: never the latest value by default.

→

2

## Code back

Written for the course in the style of an AI answer. It runs.

→

3

## Test first

Keep one date only, ask for the next, assert the error. It fails.

→

4

## Fix

WHERE key = ? AND as_of = ?, as in agentkit.recall.

**Today: recall_for_date() sorts the key's rows newest first and returns the first, whatever date was asked.**

*Code that runs is not the same as code that is right.*

# What you can do now

- An agent run's state is its messages
- A session sends the conversation again
- A desk memory keeps values with dates
- check_figures catches a remembered number
- History is part of every request's key

**Week 3, Day 13** Evals